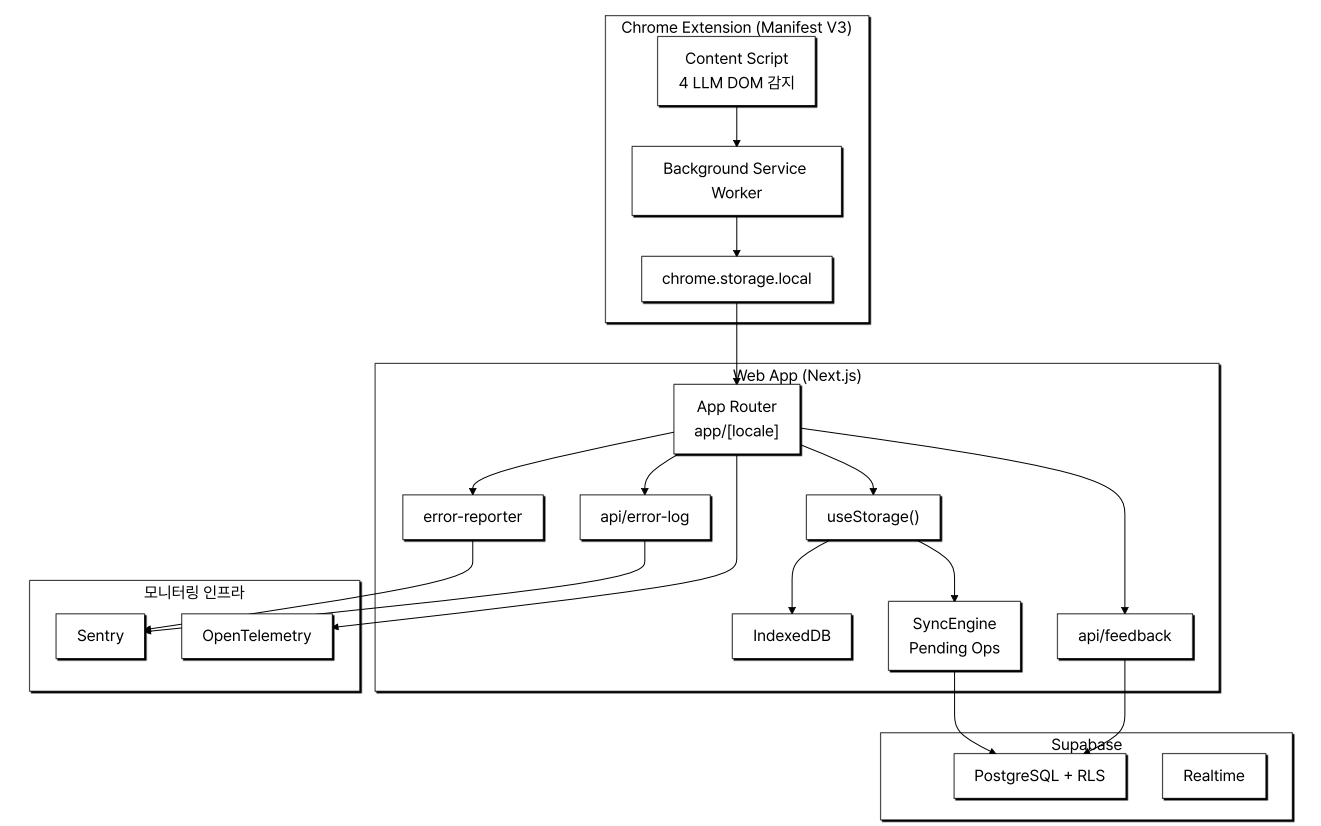


Portfolio

[MINDGRAPH] - AI 지식 캡처 & 그래프 시각화

ChatGPT·Gemini·Claude·Grok 4개 LLM 서비스의 답변을 hostname 자동 감지로 캡처해 의미 단위로 묶고 Cytoscape.js로 시각화하는 Chrome Extension·Next.js 웹앱입니다. 도메인 getmindgraph.com을 등록한 출시 전 단계의 1인 LLM 솔루션 프로토타입입니다. 출시 후 들어올 사용자 피드백을 다음 sprint로 곧장 이어 붙이려고, 사용자 피드백·에러 채널과 모니터링 인프라 (PostHog·Sentry·OpenTelemetry·api/feedback·api/error-log)를 출시 전에 먼저 갖췄습니다. Claude Code 위에 9개 혹은 plan·build·qa·ship 4 phase /sprint 워크플로우·4중 SSOT 자동 동기를 직접 설계했습니다 (Case 1에서 상세히 설명합니다).

전체적인 아키텍처



- **Architecture:** 사용자가 만나는 도구(Chrome Extension)와 통합 허브(Next.js 웹앱)를 분리하고, 사용자 신고·에러·피드백을 받을 라우트와 모니터링(Sentry·OpenTelemetry) 채널을 같은 코드 베이스에 사전 구축해 출시 후 1인 운영자가 응대와 솔루션 개선을 한 곳에서 처리할 수 있도록 설계했습니다.

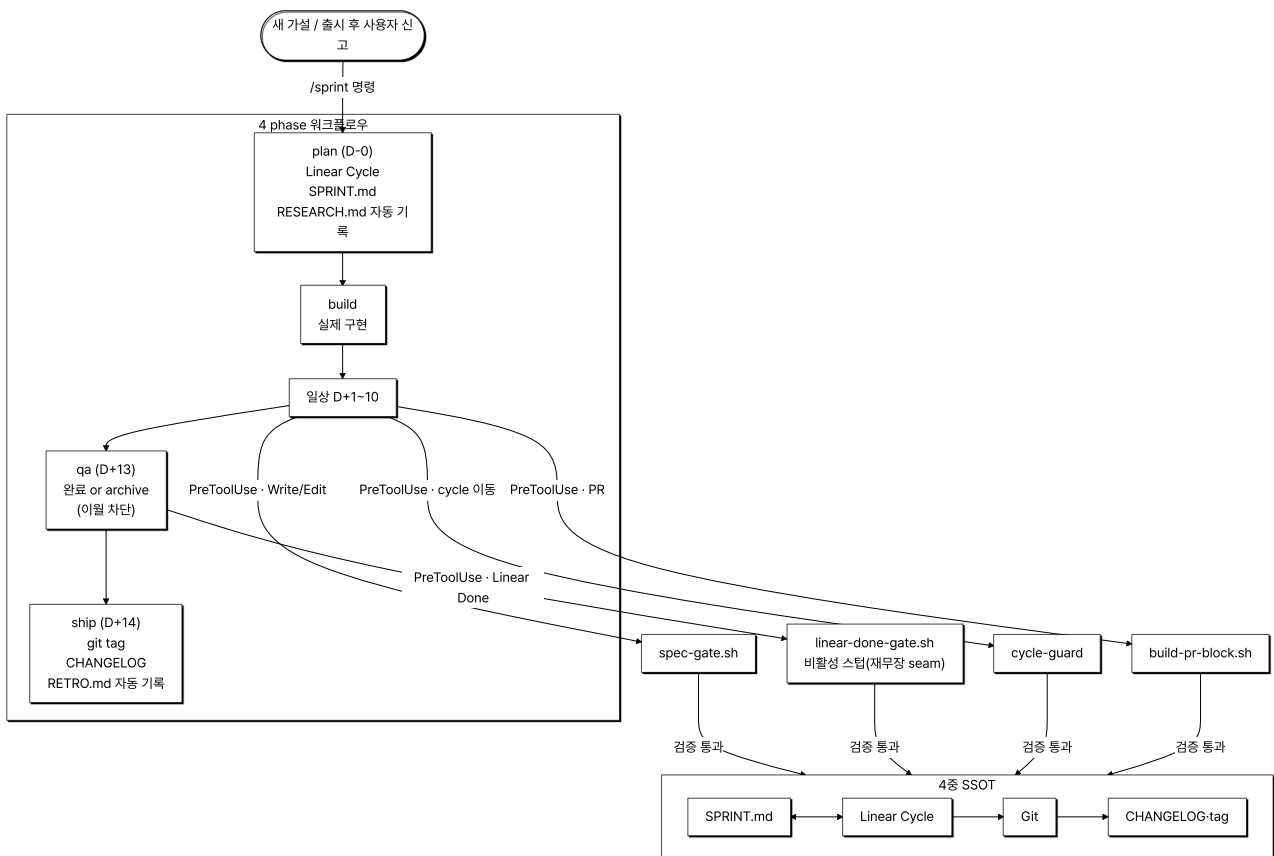
Case 1. 출시 후 1인 운영을 가능하게 할 AI 워크플로우 시스템 사전 설계

1. 문제 원인

- 기획·구현·QA·출시 후 응대 전 구간을 한 사람이 책임지는 LLM 솔루션 프로토타입에서, AI와의 대화가 세션 간 초기화되어 같은 규칙·교정을 반복 설명하는 비용이 솔루션 안정화 시간을 갉아먹었습니다.
- AI가 사용자 확인 없이 commit·이슈 완료 같은 자율 행동을 해서 검증 안 된 변경이 main 브랜치에 그대로 머지되는 사고 위험이 컸습니다.

- 코드·이슈·문서·배포 기록 네 시스템이 같은 정보를 중복 관리하면, 출시 후 사용자 신고가 들어왔을 때 현재 상태 파악을 위해 네 곳을 수동 대조해야 해 응대가 지연됩니다.

2. 해결 과정



- **9개 혹은 자동 개입:** 도구 호출 시점에 자동 실행되는 9개 혹은을 배치했고, 차단 게이트 3개(**spec-gate.sh** · **build-pr-block.sh** · **cycle-guard**)와 보조 6개(**linear-done-gate.sh** [현재 비활성 스텝] · **stale-warn.sh** · **reuse-suggest** · **integrity-sync.sh** · **context-pack-stale.sh** · **worktree-env-symlink.sh**)로 분류했습니다.
- **/sprint 4 phase:** plan(Linear Cycle·SPRINT.md·RESEARCH.md 자동 기록)·build(실제 구현)·qa(미완료 이슈 완료 또는 archive 강제, 이월 옵션 mechanism 차단)·ship(git tag·CHANGELOG·RETRO.md 자동 기록)을 단일 명령으로 연결하고 phase 진입 전 이전 산출물 존재 여부를 자동 검증했습니다. 옛 6단계 중 research·retro만 plan의 RESEARCH.md 기록과 ship의 RETRO.md 기록으로 흡수하고 build는 별도 phase로 유지했습니다.
- **4중 SSOT 동기:** SPRINT.md(문서)·Linear(이슈)·Git(코드)·CHANGELOG(배포 기록)에 역할을 하나씩 부여하고 **/sprint** 가 단계마다 자동 동기화해, 출시 후 사용자 신고가 들어왔을 때 단일 진실 원천으로 현재 상태를 파악할 수 있는 동선을 미리 마련했습니다.
- **부서·에이전트 분리 + context-map 자동 라우팅:** 7개 부서(design·dev·docs·marketing·ops·product·qa)와 9개 에이전트(ceo·frontend/backend·qa-engineer·product-manager·ui-ux-designer·marketing-strategist·ops-engineer·knowledge-logger)별로 **department/{팀}/CLAUDE.md** 와 **docs/ lifecycle** 폴더를 분리하고, **RULES/context-map.md** + **SYSTEM/schemas/code-doc-mapping.yaml** 로 task_type별 분류(new_feature·ui_change·api_change·bug_fix·refactor 등)과 코드 path glob(예: **storage/**** 변경 시 **DATABASE_SCHEMA.md** · **BACKEND.md**)을 must-read doc에 자동 매핑했습니다. CEO 에이전트가 워커에 위임할 때 합집합 5개 내외 doc만 prompt에 첨부되어, 출시 후 사용자 신고 응대 시 LLM 응답 정확도와 응대 속도를 미리 확보했습니다.

3. 결과

- **성과:** 이슈 등록·SPRINT.md·CHANGELOG·Git 태그·Linear Done 같은 sprint 라이프사이클 반복 작업을 9개 혹은 **/sprint** 4 phase로 자동화해, 출시 후 1인 운영자가 사용자 응대·솔루션 개선에 쓸 시간을 미리 확보했습니다.

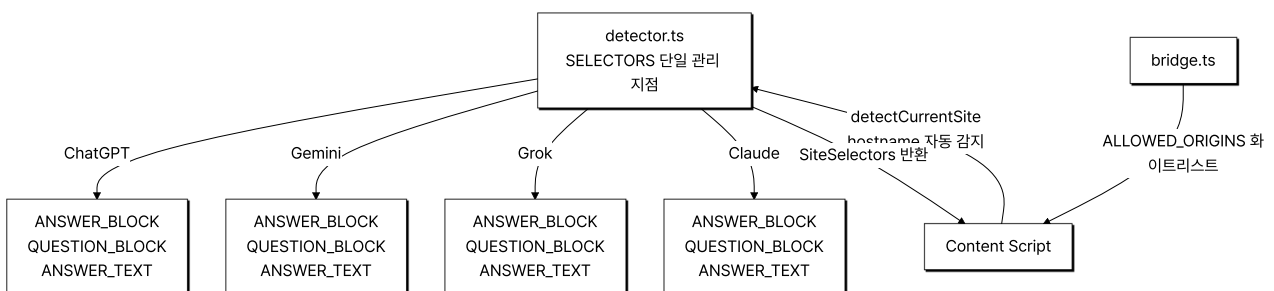
- **배운 점:** 9개 혹은 4 phase /sprint-4중 SSOT를 코드로 굳혀 두어, 새 작업마다 같은 규칙을 프롬프트로 다시 설명하지 않아도 sprint 라이프사이클이 자동으로 돌아가게 했습니다.

Case 2. 외부 LLM 서비스 DOM 변경에 끌려다니지 않는 통합 레이어 설계

1. 문제 원인

- 솔루션이 ChatGPT·Gemini·Grok·Claude 4개 외부 LLM 서비스에 의존하는 구조라, 출시 후 각 서비스 측 DOM이 공지 없이 바뀌면 그대로 사용자가 만나는 솔루션 장애로 이어질 위험이 컸습니다.
- 출시 후 사용자 신고 한 건이 들어오면 1인 운영자가 곧장 응대해야 하는데, DOM 변경 대응에 시간을 다 쓰면 다음 개선을 시작할 수 없는 구조였습니다.
- 초기에는 서비스별 선택자가 여러 파일에 분산되어 DOM 변경 한 건에 다수 파일 수정이 필요했습니다.

2. 해결 과정



- **선택자 단일 지점:** 4개 LLM 서비스의 모든 DOM 선택자를 **detector.ts** 의 **SELECTORS** 상수 객체로 모아, DOM 변경 시 이 상수만 수정하면 Content Script 전체에 일괄 적용되도록 했습니다.
- **hostname 자동 감지:** **window.location.hostname** 기반 **detectCurrentSite()** 함수가 페이지 식별과 올바른 선택자 반환을 자동 처리해, 사용자가 서비스를 수동 선택할 필요가 없습니다.
- **MV3 CSP 제약 준수:** **eval()** 사용 불가·외부 스크립트 동적 주입 불가 같은 Manifest V3 보안 정책 안에서 **chrome.runtime.sendMessage** 를 통한 메시지 통신을 구현했습니다.
- **postMessage origin 화이트리스트:** 웹앱과 Extension 사이 Extension 카트 데이터(저장한 LLM 답변 아이템) 릴레이는 **bridge.ts** 의 **ALLOWED_ORIGINS** 화이트리스트로 origin을 검증해 허가되지 않은 origin이 카트 데이터를 가져갈 수 없게 차단했습니다(**SECURITY_POLICY.md §8**).

3. 결과

- **성과:** 4개 LLM 서비스의 DOM 변경에 대응할 때 수정 범위를 **detector.ts** 상수 한 곳으로 한정해 출시 후 응대 시간을 미리 줄였고, origin 화이트리스트로 Extension과 Web 사이 카트 데이터 전달 보안을 함께 처리했습니다.
- **배운 점:** 4개 LLM 서비스 DOM 선택자를 **detector.ts SELECTORS** 한 곳에 모아 두니, DOM 변경 한 건 대응에 필요한 수정 파일 수가 한 개로 줄었습니다.

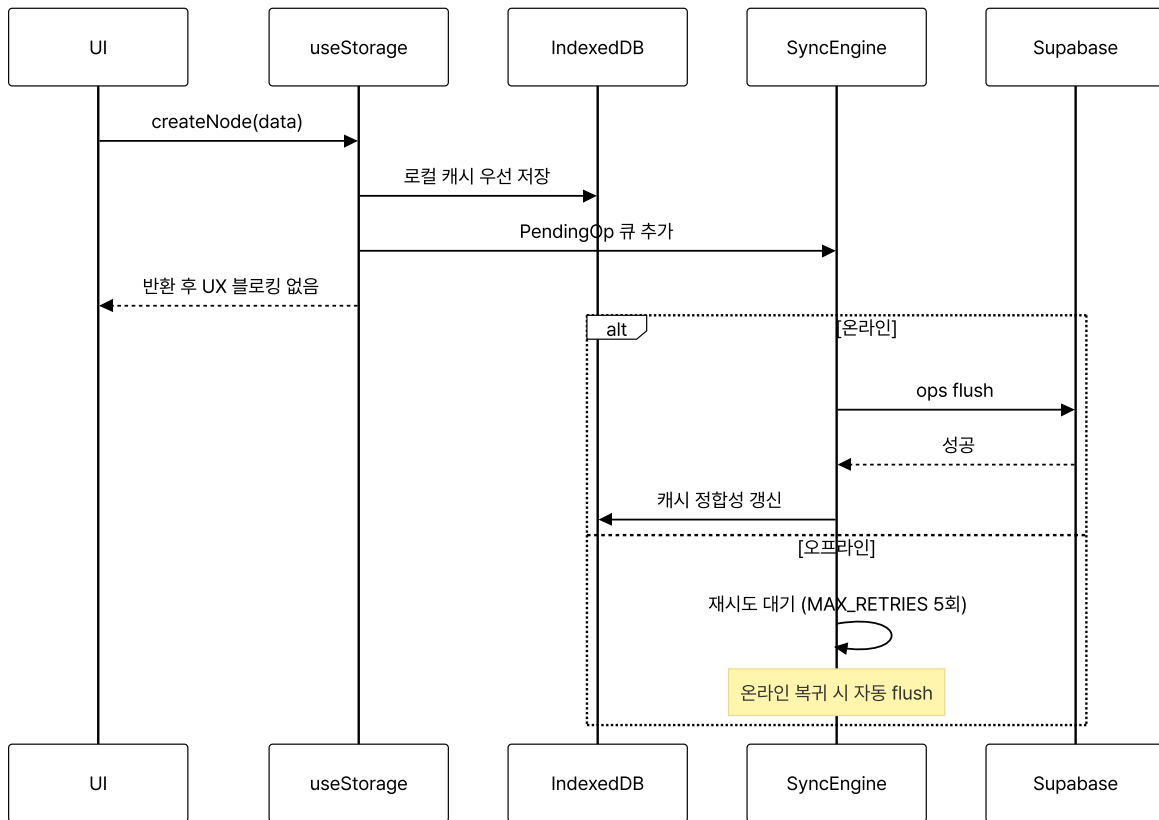
Case 3. 현장 네트워크 단절 사용자를 위한 오프라인 우선 캐시 설계

1. 문제 원인

- 사용자가 캡처를 가장 많이 시도할 곳은 카페·지하철·이동 중이고, 출시 후 응대해야 할 환경 자체가 불안정한 네트워크일 가능성이 높았습니다.
- 초기 구조가 Supabase 직접 쓰기였기 때문에 네트워크 단절 시 데이터가 사라지는 구조였고, 1인이 책임지는 LLM 솔루션 신뢰도가 출시 즉시 떨어질 수 있다고 봤습니다.

- Supabase RLS 정책 적용 중 오프라인 상태의 인증 토큰 만료 변수까지 함께 처리해야 해, IndexedDB 캐시·Pending Ops 큐·재시도 정책을 묶어 한 번에 풀어야 하는 사전 안정화 과제였습니다.

2. 해결 과정



- 읽기 경로:** `useStorage()` 혹은 항상 IndexedDB 캐시에서 먼저 응답해 네트워크 상태와 무관하게 UI가 갱신되도록 했습니다.
- 쓰기 경로:** 로컬 캐시에 먼저 쓴 뒤 `syncToCloud()` 가 `addPendingOps` 로 큐에 기록하고 `flushPendingOps` 를 await 없이 호출하는 Write-Behind 방식으로 설계해, 오프라인 상태에서도 사용자가 작업을 이어갈 수 있게 했습니다.
- 재시도 정책:** `sync-engine.ts` 의 `MAX_RETRIES = 5` 상수로 실패한 op의 재시도 한도를 두어 재시도 비용 폭발을 차단했습니다.
- Realtime 통합:** Supabase Realtime 채널 수신 시 `use-sync-manager` 가 `refreshCache()` 를 호출해 멀티 디바이스 변경을 캐시에 반영했습니다.

3. 결과

- 성과:** 오프라인 상태에서 노트 CRUD가 동작하고 온라인 복귀 시 Pending Ops 큐가 자동 flush되어, "캡처가 사라진다" 시나리오를 출시 전에 차단해 출시 후 응대 목록에서 빼냈습니다.
- 배운 점:** IndexedDB 우선 쓰기·Pending Ops 큐(MAX_RETRIES=5)·Realtime refreshCache·재시도 정책 네 가지를 묶어, 오프라인 CRUD 후 온라인 복귀 시 큐가 자동 flush되게 했습니다.

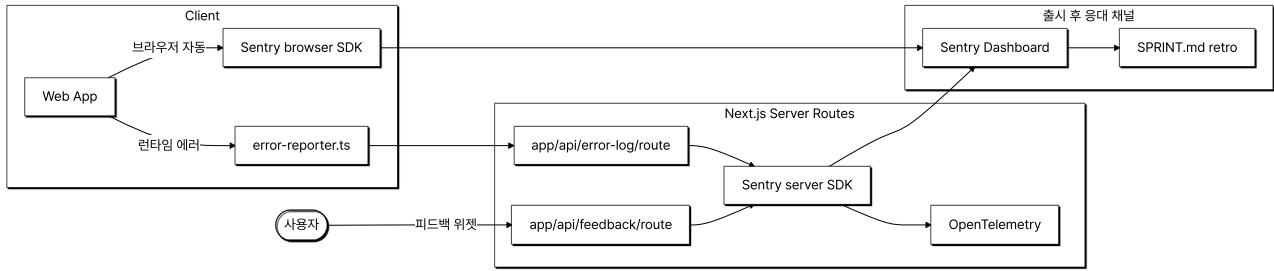
Case 4. 출시 후 사용자 에러·피드백 응대 인프라 사전 설계

1. 문제 원인

- 1인 운영자가 출시 후 사용자 에러와 신고를 수동으로 받으면 응대 지연·누락이 생기고, 어떤 사용자가 어디서 막혔는지 시스템 안에서 추적할 방법이 없으면 응대 자체가 어려웠습니다.
- 클라이언트 에러(JS 런타임 예외)는 사용자만 보고 서버 측에 도달하지 않으면 출시 후 운영자가 인지 자체를 못 하는 구조라, 인지 채널을 출시 전에 만들어 줘야 한다고 봤습니다.

- 사용자 피드백을 받을 채널을 출시 전에 만들어두지 않으면 출시 후 발생할 사용자 사고가 다음 sprint plan에 반영될 통로 없이 누락됩니다.

2. 해결 과정



- **에러 리포터 클라이언트:** `lib/error-reporter.ts` 가 클라이언트 런타임 예외를 감지해 `app/api/error-log/route.ts` 로 전송하고, 같은 에러를 Sentry browser SDK가 별도로 직접 받는 이중 수집 채널을 사전 구축했습니다.
- **에러 로그 라우트:** `app/api/error-log/route.ts` 가 클라이언트 측 에러를 서버에서 받아 Sentry server SDK에 위임해, 출시 후 운영자가 한 화면에서 확인할 수 있도록 동선을 설계했습니다.
- **피드백 라우트:** `app/api/feedback/route.ts` 가 사용자 피드백 위젯에서 받을 입력을 Sentry feedback으로 연동해, 사용자 응대 채널과 에러 응대 채널을 한 곳에 묶어 출시 후 응대 인프라를 사전 구축했습니다.
- **OpenTelemetry 통합:** `instrumentation.ts` · `instrumentation-client.ts` 에 OpenTelemetry SDK를 두고 OTLP exporter로 로그를 통합 수집해, 출시 후 시스템 흐름을 사용자 단위로 추적할 수 있는 채널을 사전 마련했습니다.
- **응대 결과 다시 sprint로:** Sentry에서 받을 사용자 사고를 `/sprint retro` 단계에서 다음 plan 입력으로 SPRINT.md에 기록할 동선을 사전 연결해, 출시 후 같은 문제가 반복되지 않도록 설계했습니다.

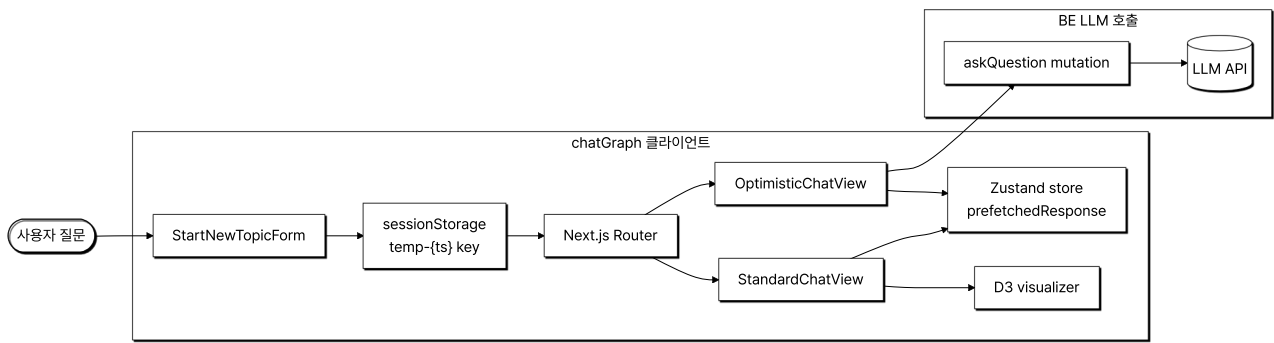
3. 결과

- **성과:** 클라이언트 에러(error-reporter)·서버 에러(api/error-log)·사용자 피드백(api/feedback)·OpenTelemetry 로그 4개 채널을 Sentry 한 곳에 묶어, 출시 후 응대 대상이 단일 화면에 모이고 응대 결과를 다음 sprint plan으로 이어 붙일 수 있는 사이클을 사전 구축했습니다.
- **배운 점:** error-reporter·api/error-log·api/feedback·OpenTelemetry 네 채널을 Sentry 한 곳으로 모으고 retro 단계에서 SPRINT.md로 옮기는 동선을 코드로 연결한 결과, 사용자 사고가 다음 plan 입력으로 누락 없이 이어지는 흐름이 확보됐습니다.

[CHATGRAPH] - AI 대화 시각화 플랫폼

기존 선형 채팅 UI에서 한 갈래 흐름만 살아남고 도중에 떠올린 분기 질문·맥락이 휘발되어 사용자가 이전 맥락을 다시 짜맞춰야 하는 비용이 컸습니다. chatGraph는 LLM과의 대화를 꼬리물기 형태의 그래프로 시각화하여 분기와 깊이를 보존하는 LLM 응답 통합 솔루션입니다. 본인은 4인 팀의 FE 한 명으로 참여해 LLM 응답 대기 동안의 사용자 흐름과 응답을 시각화로 연결하는 라이프사이클을 담당했습니다.

전체적인 아키텍처



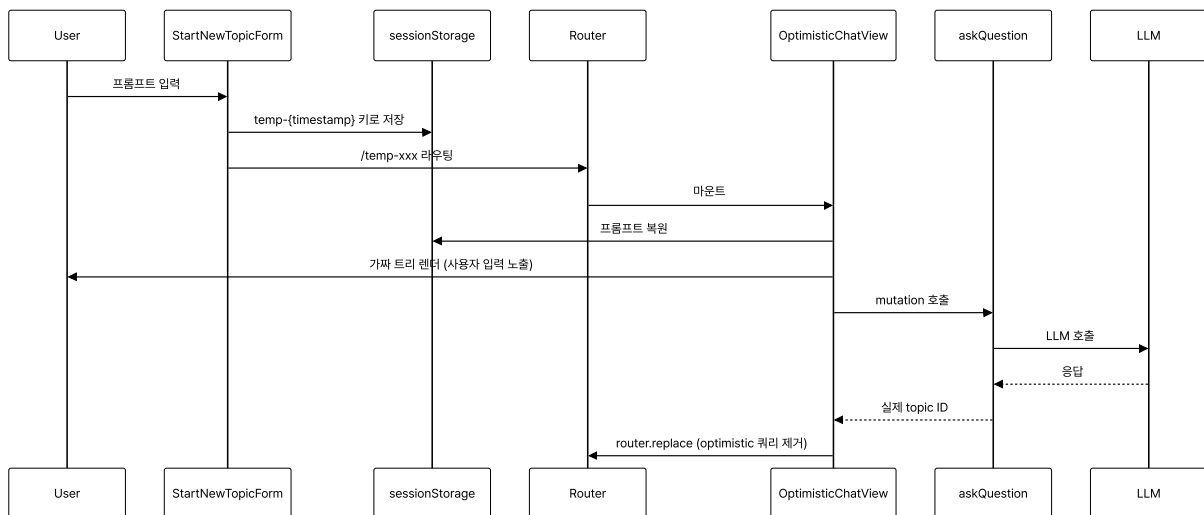
- **Architecture:** 사용자 입력을 받아 임시 화면을 곧장 노출(Optimistic)한 뒤 백엔드 LLM을 호출하고 응답 도착 후 실제 화면으로 정착하는 4단 클라이언트 흐름. LLM 응답 대기 동안 사용자가 보는 화면을 클라이언트 단에서 책임지는 구조.

Case 1. LLM 응답 대기 동안 사용자 흐름 단절을 막는 Optimistic 라우팅

1. 문제 원인

- 사용자가 첫 질문을 입력하면 LLM 응답이 도착할 때까지 화면이 멈춘 듯한 경험이 사용자 몰입을 끊었습니다. 외부 LLM API 응답은 수 초 단위가 일반적이라, 대기 시간이 길수록 사용자는 첫 화면에서 멈춘 느낌을 받습니다.
- 단순 토스트·스피너 방식은 새 토픽이 생성될 URL을 미리 보여주지 못해 뒤로가기·새로고침 동작이 어색했고, LLM 응답 대기 중 사용자가 페이지를 잘못 닫으면 입력이 사라졌습니다.

2. 해결 과정



- **임시 ID + sessionStorage:** StartNewTopicForm 에서 `temp-{Date.now()}` 형태의 임시 ID를 만들어 sessionStorage에 프롬프트를 보관(`use-start-new-topic.ts`)하고, 곧장 임시 경로로 push해 사용자에게 자기 입력이 그려진 화면을 노출합니다.
- **가짜 트리 복원:** 이동한 경로에서 `useOptimisticTopicData` 훅이 sessionStorage에서 프롬프트를 복원해 한 단계짜리 트리를 렌더링 하면서 동시에 실제 mutation을 시작합니다.
- **LLM 응답 정착:** mutation 성공 시 Zustand 스토어에 응답을 채우고 `router.replace` 로 optimistic 쿼리 파라미터를 제거해 실제 topic ID 경로로 자연스럽게 전환합니다.

3. 결과

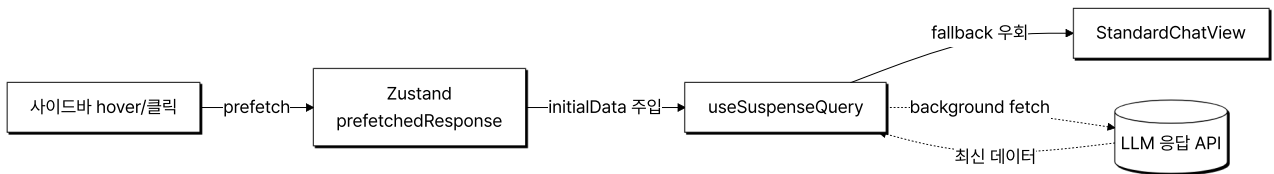
- **성과:** LLM 응답까지의 대기 시간 동안에도 사용자가 자기 입력이 그려진 화면을 보고 있어, LLM 솔루션의 첫 인상이 외부 응답 시간에 의존하지 않게 만들었습니다. 새로고침·공유 링크도 정상 동작합니다.
- **배운 점:** 백엔드 응답이 늦어도 사용자가 빈 화면을 보지 않도록 sessionStorage 임시 저장·임시 ID 라우팅·mutation 정찰을 한 흐름으로 묶었습니다.

Case 2. LLM 응답 프리패치로 토픽 탐색 응대 매끄럽게

1. 문제 원인

- 사이드바에서 다른 토픽을 클릭하면 useQuery가 새로 호출되며 페이지가 잠시 빈 상태로 보였습니다. LLM 솔루션의 누적 응답이 늘어날수록 토픽 사이를 이동하는 사용자 흐름이 핵심인데, 매번 빈 화면 fallback이 났습니다.
- App Router Suspense fallback 노출 시 깜빡임이 발생했고, 응답량이 많은 토픽일수록 사용자 체감 지연이 커져 사용자가 솔루션을 신뢰하기 어려웠습니다.

2. 해결 과정



- **사이드바 prefetch:** 사이드바에서 토픽 응답을 미리 받아 Zustand 스토어의 **prefetchedResponse** 슬롯에 저장합니다.
- **initialData 우회:** 페이지 진입 시 **useSuspenseQuery**의 initialData에 동일 topicId의 프리패치 응답이 있으면 그대로 사용 (**use-topic-data.ts**)하고, 없을 때만 Suspense fallback을 거치도록 분기했습니다.
- **백엔드 응답 변환:** **transformApiDataToViewData** 어댑터가 백엔드 응답을 트리 형태로 변환해 클라이언트에 주입 (**use-tree-data.ts**)합니다.
- **잔여 데이터 정리:** 사용한 프리패치 데이터는 effect cleanup에서 정리해 다음 토픽 전환 시 잔여 데이터가 섞이지 않도록 했습니다.

3. 결과

- **성과:** 사이드바 기반 토픽 탐색에서 빈 화면 노출 없이 LLM 응답 트리가 그려져, 사용자가 누적된 LLM 응답 사이를 빠르게 오가는 탐색 경험을 갖췄습니다.
- **배운 점:** 토픽 전환 시 빈 화면이 뜨지 않도록 TanStack Query initialData와 Zustand prefetch 슬롯을 같은 키로 맞춰 두 캐시 계층이 충돌하지 않게 조립했습니다.

Case 3. D3 데이터 조인으로 LLM 응답 시각화 부분 갱신

1. 문제 원인

- 팀원이 도입한 D3 Force Simulation 시각화는 데이터가 바뀔 때마다 **selectAll("*").remove()**로 SVG 전체를 지우고 hierarchy.simulation을 처음부터 다시 구성했습니다. LLM 응답이 한 노드만 추가돼도 그래프 전체가 재생성됐습니다.
- 매 갱신마다 시뮬레이션을 새로 만드니 기존 노드의 좌표가 초기화돼 화면 전체가 다시 흩어졌다 모이고, 토픽이 바뀌거나 트리가 갱신될 때마다 이전에 사용자가 보던 배치가 유지되지 않았습니다.

2. 해결 과정



- **마운트 1회 프레임 재사용:** svg 프레임·defs(필터)·zoom·레이어·forceSimulation·tick은 마운트 시 한 번만 생성하고, 이후 갱신에서는 재생성하지 않고 그대로 재사용합니다. `selectAll("*").remove()` 는 언마운트 cleanup에서만 1회 호출합니다.
- **노드 id 키 데이터 조인:** data/currentPath가 바뀌면 `.data(nodes, (d) => d.data.id).join(...)` 으로 추가·변경·삭제된 노드만 enter/update/exit 처리해 DOM에 부분 반영합니다. forceLink links와 simulation.nodes도 함께 갱신합니다.
- **좌표 보존 후 재가열:** 기존 노드는 `positionsRef` 에 저장한 좌표를 복원해 자리를 유지하고, 데이터 변경 후 `alpha(0.3).restart()` 로 시뮬레이션만 재가열해 새 노드를 부드럽게 안착시킵니다. 키 기반 조인 덕분에 StrictMode 이중 마운트에서도 정리가 안전했습니다.
- **팀 역할 분담:** 색상 알고리즘·호버 desaturate는 팀원 코드를 유지하고, 본인은 React 통합과 D3 데이터 조인 재설계를 맡았습니다.

3. 결과

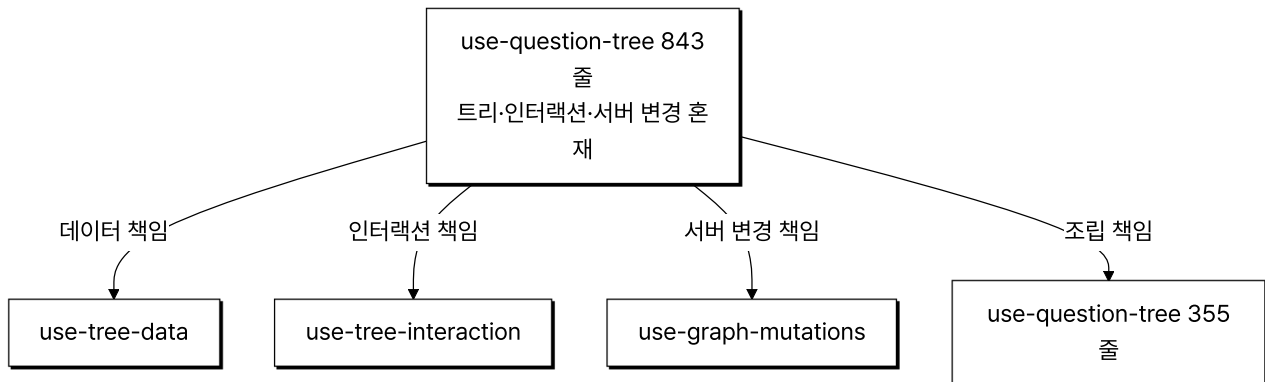
- **성과:** LLM 응답이 추가될 때 그래프 전체를 다시 그리지 않고 바뀐 노드만 갱신하며, 기존 노드는 좌표를 유지해 토픽 전환·트리 갱신에도 사용자가 보던 배치가 그대로 남았습니다.
- **배운 점:** React 선언형 트리와 D3 명령형 SVG가 한 컴포넌트에 공존하는 구간에서, 전체 파괴·재생성 대신 노드 id를 키로 한 enter/update/exit 조인과 좌표 보존으로 변경분만 반영하는 방식을 익혔습니다.

Case 4. LLM 응답 흐름을 받아낼 모듈 책임 분리 (843줄에서 355줄로 축소)

1. 문제 원인

- LLM 응답을 다루는 use-question-tree 혹은 트리 상태와 인터랙션·서버 변경을 모두 끌어안아 843줄까지 비대해졌고, LLM 응답 형식이 바뀔 때마다 어디를 고쳐야 할지 영향 범위 파악이 어려웠습니다.
- 페이지·기능·공통 코드 경계가 모호해 컴포넌트끼리 순환 import가 반복 발생했습니다. 이대로 두면 새 LLM 기능을 추가할 때마다 같은 결함이 누적됩니다.

2. 해결 과정



- **4-책임 분리:** 단일 혹은 들고 있던 책임을 트리 데이터(`use-tree-data`)·인터랙션 상태(`use-tree-interaction`)·서버 변경(`use-graph-mutations`)으로 분리하고 최상위에서 합성합니다.
- **Feature-First 디렉토리:** app은 라우팅, features는 도메인, views는 조립, shared는 공통이라는 네 책임으로 디렉토리를 재정의하고 import 방향을 단방향으로 정비했습니다.
- **점진적 리팩토링:** 본인 단독 커밋 3건에 걸친 누적 리팩토링으로 진행했고, 단일 PR이 아닌 점진적 분해로 운영 중 위험을 분산했습니다.

3. 결과

- **성과:** 핵심 혹은 843줄에서 355줄로 축소, 인접 페이지 컴포넌트도 186줄에서 59줄로 정리해 LLM 응답 형식 변경 시 영향 범위를 한 책임 단위로 좁혔습니다.
- **배운 점:** 843줄짜리 단일 혹은 트리 데이터·인터랙션·서버 변경 세 책임으로 쪼개 두니, 이후 LLM 응답 포맷이 바뀌었을 때 수정 범위가 한 파일로 좁혀졌습니다.